

Module 7 Week A — Applied Lab: Fine-Tune DistilBERT for Sentiment Classification

In this lab you will fine-tune DistilBERT on the AARSynth app reviews dataset, evaluate the resulting classifier, push the model to your free Hugging Face Hub account, and write a one-page evaluation report. This is the first concrete deliverable of Module 7's "fine-tuned model + evaluation report" — Week A produces a fine-tuned classifier and a per-task report; Week B will integrate it with QA and summarize it into a comparative analysis.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

What You Will Produce

Committed to your repo:

- `lab.py` — your implementation of the fine-tuning pipeline.
- `metrics.json` — accuracy, macro-F1, and per-class F1 / precision / recall on the test split.
- `predictions.csv` — per-row test set predictions with the predicted-class probability and the full per-class probability distribution.
- `confusion_matrix.csv` — the test-split confusion matrix as a square CSV (rows = true label, columns = predicted label).
- `evaluation-report.md` — a one-page report (the structure is given below).

Local-only and not committed:

- `model/` — your fine-tuned checkpoint. The starter `.gitignore` excludes this directory because DistilBERT checkpoints are ~265 MB, which exceeds GitHub's 100 MB hard file limit. The model is produced locally and pushed to Hugging Face Hub instead.

Published to Hugging Face Hub:

- Your fine-tuned model** at `https://huggingface.co/<your-username>/m7-app-review-sentiment` — this is the model you (and the Integration Task later in the week) load via `AutoModelForSequenceClassification.from_pretrained(...)`.

Setup

1. Hugging Face account + token (one-time)

If you don't already have an HF account, create one at huggingface.co/join (free). Then create a token with **write** scope:

- Go to huggingface.co/settings/tokens.
- Click **New token**.
- Give it a name (e.g., `aispire=14005`), select **Write** as the role, and create.
- Copy the token. You'll paste it during `huggingface-cli login` below.

Security note: the HF token is separate from your GitHub Personal Access Token (PAT). Don't confuse them. Don't commit either of them to a repo.

2. Accept and clone the lab repo

1. **Accept the assignment**

2. Clone:

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. Install dependencies (use a fresh venv if you hit version conflicts):

```
pip install -r requirements.txt
```

The default `torch` wheel on Linux is CUDA-enabled (~2 GB). If you're on a CPU-only machine and don't want the GPU build, install torch from the CPU index **first**, then run the line above:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements.txt
```

On macOS (including Apple Silicon), the default wheel is already CPU/MIPS — no extra step needed.

4. Authenticate to Hugging Face Hub (one time per machine):

```
huggingface-cli login
```

Paste your token when prompted. The CLI saves it to `~/.cache/huggingface/token` so you don't need to log in again.

5. **Verify your token has Write scope** before training. Training takes ~60–90 minutes; a read-only token will fail at the very end (`push_to_hub` → `403 Forbidden`) and force you to redo the run. Two ways to check:

- Visit huggingface.co/settings/tokens — the token you just created should show **Write** in the **role** column.
- Or run `huggingface-cli whoami` (confirms login) and then look at the role on the settings page.

If the role is **Read**, regenerate the token with **Write** selected and re-run `huggingface-cli login`.

3. Create your working branch

```
git checkout -b lab-7a-finetune-sentiment
```

All implementation work goes in `lab.py`. The starter file contains function signatures and docstrings — implement each as described below.

What You're Building

`lab.py` is one Python script that runs the full fine-tuning pipeline end-to-end. The structure mirrors the conceptual flow from the Reading: `load data` → `tokenize` → `configure` → `train` → `evaluate` → `save` → `push to Hub`.

You will implement the following functions. `make_training_args` and `compute_metrics` play the same role as in the drill; `tokenize_dataset` is the same step but with a small signature change (see Task 2). The lab adds `prepare_dataset` (the drill's `make_dataset` renamed for clarity), `train_classifier`, `evaluate_classifier`, and the `orchestrating` `main()`:

```
get_data_path()           # provided helper
prepare_dataset(...)       # TODO
tokenize_dataset(...)      # TODO (same as drill)
make_training_args(...)    # TODO (same as drill)
compute_metrics(...)       # TODO (same as drill)
train_classifier(...)      # TODO (new)
evaluate_classifier(...)    # TODO (new)
main()                    # TODO (orchestration)
```

You're welcome to copy your drill solutions for `make_training_args` and `compute_metrics`. They do the same thing here. `tokenize_dataset` has a small signature change in the lab — see Task 2 below.

The starter `lab.py` includes `get_data_path()`, which returns `os.environ.get("DATA_PATH")`, if set, otherwise the default path to the curated training CSV at `data/app_reviews_train.csv`. **Don't modify this helper** — the autograder uses it to switch between the smoke fixture (CI) and the real dataset (your local runs).

Task 1: Prepare the Dataset

Implement `prepare_dataset(data_path, test_size=0.2, seed=42)`:

- Load the CSV at `data_path` with pandas, then convert via `Dataset.from_pandas(df, preserve_index=False)`.
- Split into train and test using the passed `test_size` and `seed`.
- Return the resulting `DatasetDict`.

Both the smoke fixture (`fixtures/tiny_app_reviews.csv`) and the real curated dataset (`data/app_reviews_train.csv`) ship as CSVs in the repo, so this is single-path CSV loading.

Task 2: Tokenize and Train

`tokenize_dataset` is the same as the drill in spirit, but the lab passes in a pre-loaded tokenizer object instead of a name so `main()` doesn't load the tokenizer twice. Signature: `tokenize_dataset(ds_dict, tokenizer, max_length=128)`. Inside, call `ds_dict.map(tokenize_fn, batched=True)` where `tokenize_fn` calls the passed-in `tokenizer` with `truncation=True, max_length=max_length`. No padding here — `DataCollatorWithPadding` pads dynamically at training time.

Then implement `train_classifier`:

```
def train_classifier(
    tokenized_ds: DatasetDict,
    model_name: str,
    training_args: TrainingArguments,
    tokenizer,
    num_labels: int,
) -> Trainer:
    """
    Fine-tune a pre-trained model on the tokenized dataset.

    Returns the trained Trainer (with the fine-tuned model accessible as trainer).
    """
    # TODO: load model with AutoModelForSequenceClassification.from_pretrained(model_name)
    # TODO: build a DataCollatorWithPadding(tokenizer=tokenizer)
    # TODO: construct a Trainer with model, args, train/eval datasets, tokenizer,
    # TODO: call trainer.train()
    # TODO: return trainer
```

Common pitfalls:

- Forgetting `num_labels`** — the model's classification head must match your dataset. AARSynth app reviews has 3 classes: `pass`, `neutral`, `negative`.
- Not passing `data_collator`** — without it, batches with mixed-length sequences will fail.
- Saving the model in the wrong place** — `Trainer` saves checkpoints into `output_dir` (which you set in `TrainingArguments`). The final saved model lives there.

Task 3: Evaluate the Classifier

Implement `evaluate_classifier`. Outcome 5 of this module asks for the full diagnostic set — accuracy, precision, recall, macro-F1, and a confusion matrix — so the returned dict carries five keys and the confusion matrix gets persisted in Task 4.

```
def evaluate_classifier(trainer: Trainer, tokenized_test) -> dict:
    """
    Evaluate the fine-tuned classifier on the test split.

    Returns a dict with keys:
    - accuracy          (float)
    - macro_f1          (float)
    - per_class_f1      (label_name: float, ...)
    - per_class_precision (label_name: float, ...)
    - per_class_recall   (label_name: float, ...)

    The three per-class dicts are keyed by the string label name (e.g., "positive") read from model.config.id2label.
    """
    # TODO: get predictions on tokenized test using trainer.predict
    # TODO: argmax the logits to get predicted class indices
    # TODO: compute accuracy (accuracy_score) and macro_f1 (f1_score, average="macro")
    # TODO: compute per-class F1 with f1_score(..., average=None)
    # TODO: compute per-class precision with precision_score(..., average=None, zero_div=0)
    # TODO: compute per-class recall with recall_score(..., average=None, zero_div=0)
    # TODO: build per_class_f1 / per_class_precision / per_class_recall dicts and
    # TODO: return all five keys
```

Use `zero_division=0` on precision and recall so a class with no predicted samples scores 0 cleanly instead of raising a `sklearn UndefinedMetricWarning`.

Important — read labels from `model.config.id2label`. Don't hard-code class names. The integration task and stretch assignments use the same pattern with different models that may have different label mappings.

The starter `main()` prints the metrics inline and writes the confusion matrix to `confusion_matrix.csv` (Task 4) so you can inspect both during development and keep the matrix as a committed artifact.

Reproducibility note. The starter `main()` calls `set_seed(42)` from `Transformers` before constructing the model. The classification head is randomly initialized each time you load the model with `AutoModelForSequenceClassification.from_pretrained(...)`, so without a fixed seed, your predictions on the test split — and therefore the three qualitative error examples you discuss in `evaluation-report.md` — would change every time you re-run `python lab.py`. With the seed fixed, the same training run produces the same predictions and the same misclassifications, which makes the error-analysis section of your report stable across reruns. The `seed` field in `TrainingArguments` covers the Trainer's internal sampling, but not the head initialization — that's why `set_seed(...)` is called separately.

Task 4: Persist Outputs

After training and evaluating, your `main()` must produce the committed artifacts. The starter includes a small helper for `predictions.csv`. You write the rest:

- `metrics.json`** — `[{"accuracy": ..., "macro_f1": ..., "per_class_f1": {...}, "per_class_precision": {...}, "per_class_recall": {...}]` from `evaluate_classifier`.
- `confusion_matrix.csv`** — the test-split confusion matrix as a CSV (rows = true label, columns = predicted label). See Task 4 instructions for the writing pattern.
- `predictions.csv`** — one row per test example with columns `text`, `label`, `predicted_label`, `predicted_probability`, plus one `prob_<label>` column per class carrying the full softmax distribution (e.g., `prob_neutral`, `prob_neutral`, `prob_positive`). The per-class columns let your `evaluation-report` surface the **gold-class probability** on misclassifications (read `prob_gold_label` for that row) and feed the Tuesday Honors calibration stretch.
- `evaluation-report.md`** — see "Writing the Evaluation Report" below.

Use `json.dump(metrics, f, indent=2)` to write `metrics.json` so it's diff-friendly.

Task 5: Push the Model to Hugging Face Hub

After saving locally, push to your HF account:

```
trainer.save_model("model")
tokenizer.save_pretrained("model")
trainer.push_to_hub("m7-app-review-sentiment")
tokenizer.push_to_hub("m7-app-review-sentiment")
```

The first call uploads the model; the second uploads the tokenizer (without it, anyone loading your model from the Hub would have to bring their own tokenizer). After both succeed, your model lives at:

```
https://huggingface.co/<your-username>/m7-app-review-sentiment
```

You will copy this URL into your evaluation report and into the Integration Task next.

If `push_to_hub` fails with an authentication error, run `huggingface-cli login` again and re-paste your token. If it still fails, your token may not have write scope — generate a new token with write scope.

Writing the Evaluation Report

Create `evaluation-report.md` with these sections:

```
## Module 7 Week A — Lab Evaluation Report

# Dataset
1-2 sentences describing AARSynth app reviews Sentences=50/Agree
(N examples, label distribution, split sizes).

## Model and hyperparameters
- Backbone: distilbert-base-uncased
- Number of labels: 3
- Learning rate, epochs, batch size, max_length, seed
- Training time (wall-clock) on your machine

## Metrics on the test split

Aggregate:

| Metric | Value |
|---|---|
| Accuracy | ... |
| Macro-F1 | ... |

Per class (read from 'metrics.json'):

| Class | F1 | Precision | Recall |
|---|---|---|---|
| Positive | ... | ... | ... |
| Neutral | ... | ... | ... |
| Negative | ... | ... | ... |

## Confusion matrix
[Insert a small markdown table or commit a screenshot. The starter includes a helper that prints a sklearn confusion_matrix for you.]

## Three qualitative error examples (one per class)
For each example, include:
- The original sentence
- Gold label
- Predicted label
- Predicted probability for the gold label
- 1-2 sentences on why you think the model got this one wrong.

## Hugging Face Hub model URL
https://huggingface.co/<your-username>/m7-app-review-sentiment
```

The qualitative error analysis is the part TAs read most carefully. Don't pick three random misclassifications — pick three that are **informative** about how the model fails. Look for:

- A neutral sentence the model called negative (or vice versa) — what cue word may have triggered it?
- A long sentence with mixed sentiment that the model classified as one extreme — did it overweight a particular phrase?
- A sentence that's hard for a human — your reasoning here calibrates expectations for production use.

Submission

- Verify that `pytest tests/ -v` runs and the autograder tests pass (the autograder runs in CI on every push).
- Commit and push the branch.
- Open a PR from `lab-7a-finetune-sentiment` into `main`.
- Wait for the autograder to complete (5–10 minutes — most of it is pip install and the smoke training run).

Your PR description must include:

- Dataset description** (1–2 sentences).
- Final hyperparameters** — learning rate, epochs, batch size, max_length, seed.
- Final metrics** — accuracy, macro-F1, per-class F1.
- Confusion matrix interpretation** (1 sentence) — which class is hardest to identify? Which two classes does the model confuse most?
- Hugging Face Hub model URL**.
- Paste your PR URL into TalentLMS → Module 7 → Applied Lab 7A to submit this assignment.

Troubleshooting

- `pip install` fails with a torch-related error** — install torch from the CPU index explicitly: `pip install torch --index-url https://download.pytorch.org/whl/cpu`, and then re-run `pip install -r requirements.txt`.
- Training runs but loss never decreases** — check that `train_dataset` and `eval_dataset` are passed to `Trainer` with the tokenized splits (a common bug is passing the `un-tokenized` dataset).
- The model is randomly initialized, warning when loading** — this is expected. The warning fires because the classification head is initialized fresh; the backbone is loaded from the pre-trained checkpoint. Train the model and the warning is no longer relevant.
- Out of memory during training** — reduce `per_device_train_batch_size` from 8 to 4, or reduce `max_length` from 128 to 96.
- `push_to_hub` fails with `403 Forbidden`** — your token doesn't have write scope. Create a new token with write scope at huggingface.co/settings/tokens.
- CI training takes way too long** — CI uses the smoke fixture (60 rows), not the real dataset. If your local code uses the real dataset path and you push that to CI, training will time out. Use `get_data_path()` consistently — don't hard-code the dataset.

Challenge Extensions

The base assignment is complete and gradable on its own. The challenge tiers are optional extensions for learners who want a deeper engagement. Each tier adds new content, nothing in a tier modifies the base submission.

Challenge Tier 1 — Hyperparameter Sweep

Re-run the fine-tune at three learning rates: `1e-5`, `5e-5`, `1e-4`, with all other hyperparameters held constant. For each run, record macro-F1 on the test split. Add a section to your `evaluation-report.md` with a table of the three results and 2–3 sentences interpreting which performed best and why.

You don't need to submit three separate `model/` directories — a single table with three rows is enough.

Challenge Tier 2 — Model Comparison: DistilBERT vs. BERT-base

Fine-tune both `distilbert-base-uncased` and `bert-base-uncased` on the same data with the same hyperparameters. Compare:

- Macro-F1 on the test split
- Wall-clock training time on your machine
- Saved model size on disk

Write a 1-page memo (`model-comparison.md`) on the cost-vs-quality trade-off. Be specific: at what point would you pay for the extra training time of BERT-base?

Challenge Tier 3 — Manual Fine-Tuning Loop Without `Trainer`

Reproduce the fine-tune using a hand-written PyTorch training loop (forward pass, loss computation, backward pass, optimizer step, periodic evaluation) — no `Trainer`, no `TrainingArguments`. Compare your manual implementation's metrics to the `Trainer` version on the same data and seed.

This tier requires understanding `DataCollator`, attention masks, optimizer schedulers, and evaluation cadence at the level of the `Trainer` source. A senior engineer should need 4+ hours. The point is not to outperform `Trainer` — it is to understand what `Trainer` is doing for you under the hood.

Add `manual_train.py` and a `manual-train-comparison.md` memo to your repo.